

# AIRLINE MANAGEMENT SYSTEM

## Database Management System (DBMS) Project

PAK Airlines — Database: pak\_airlines

---

Subject: Database Management Systems (DBMS)

Submitted To: Mam Farwa Khalid

**Team Members: Nahah Butt | Muhammad Talha**

Project Duration: March 2025 – June 2025

## 1. Introduction

---

The Airline Management System is a relational database project developed as part of the Database Management Systems (DBMS) course. The system is designed to manage the core operations of an airline, including aircraft registration, airport management, flight scheduling, fare management, and passenger seat reservations.

The database, named `pak_airlines`, is implemented on MariaDB 10.4 and managed through phpMyAdmin. It follows a normalized relational model with clearly defined primary keys, foreign key constraints, and referential integrity across all entities.

This document covers the complete database design including the entity structure, schema definitions, relationships, and a summary of the data stored in each table.

## 2. Project Objectives

---

- Design a fully normalized relational database for airline operations.
- Manage airplane types, individual aircraft, and airport landing permissions.
- Schedule flights with multi-leg support and operating weekday patterns.
- Track specific dated occurrences of each flight leg (leg instances).
- Manage multiple fare classes (Economy and Business) per flight.
- Handle passenger seat reservations with contact information.
- Enforce referential integrity through foreign key constraints.

## 3. Database Overview

---

The `pak_airlines` database consists of 9 interrelated tables. The design follows the standard relational model taught in DBMS, covering concepts such as primary keys, composite keys, foreign keys, and normalization. Below is a high-level summary of each table.

| Table Name   | Primary Key                               | Description                                                            |
|--------------|-------------------------------------------|------------------------------------------------------------------------|
| airline_type | Airplane_type_name (PK)                   | Stores aircraft type info including max seats and manufacturer company |
| airplane     | Airplane_id (PK)                          | Stores individual airplane records linked to a type                    |
| airport      | Airport_code (PK)                         | Stores airport details: name, city, and state                          |
| can_land     | Airplane_type_name +<br>Airport_code (PK) | Junction table — which airplane types can land at which airports       |
| flight       | Flight_number (PK)                        | Stores flight schedules including airline and operating weekdays       |

| Table Name       | Primary Key                                      | Description                                                                |
|------------------|--------------------------------------------------|----------------------------------------------------------------------------|
| flight_leg       | Flight_number + Leg_number (PK)                  | Defines each leg of a flight: departure & arrival airports and times       |
| leg_instance     | Flight_number + Leg_number + Date (PK)           | A specific dated occurrence of a flight leg with seat availability         |
| fare             | Flight_number + Fare_code (PK)                   | Fare classes (Economy/Business) for each flight with amount & restrictions |
| seat_reservation | Flight_number + Leg_number + Date + Seat_no (PK) | Passenger seat bookings with customer name and contact                     |

## 4. Entity-Relationship Description

The following describes the key relationships between entities in the database:

### 4.1 Airplane Type → Airplane

Each airplane belongs to exactly one airplane type (airline\_type). The Airplane\_type column in the airplane table is a foreign key referencing airline\_type. This establishes a one-to-many relationship: one type can have many individual aircraft.

### 4.2 Airplane Type ↔ Airport (Can Land)

The can\_land table is a many-to-many junction table linking airplane types to airports. It records which airplane types are permitted to land at which airports. Both columns form a composite primary key and each references its respective parent table.

### 4.3 Flight → Flight Leg

A flight can have one or more legs (e.g., a stop-over route). The flight\_leg table uses a composite primary key of Flight\_number and Leg\_number. Each leg specifies a departure airport, arrival airport, and scheduled times.

### 4.4 Flight Leg → Leg Instance

A flight leg has multiple dated instances (leg\_instance). Each row in leg\_instance represents a specific date on which that leg operated, with a specific airplane assigned and real-time departure/arrival details. This table tracks seat availability per instance.

### 4.5 Flight → Fare

Each flight has one or more fare records. The fare table stores Economy and Business class pricing along with restrictions like 'No Refund', 'Flexible', or 'Priority Boarding'.

### 4.6 Leg Instance → Seat Reservation

Passengers are booked against specific leg instances on specific dates. The seat\_reservation table stores customer name, phone number, and seat number, all linked to a specific flight, leg, and date.

## 5. Database Schema (Table Definitions)

### 5.1 Table: airline\_type

| Column Name        | Data Type    | Constraint  | Description                         |
|--------------------|--------------|-------------|-------------------------------------|
| Airplane_type_name | VARCHAR(50)  | PRIMARY KEY | Unique identifier for aircraft type |
| Max_seats          | INT          | -           | Maximum seating capacity            |
| Company            | VARCHAR(100) | -           | Manufacturer / operating company    |

### 5.2 Table: airplane

| Column Name       | Data Type   | Constraint        | Description                         |
|-------------------|-------------|-------------------|-------------------------------------|
| Airplane_id       | VARCHAR(20) | PRIMARY KEY       | Unique airplane identifier          |
| Total_no_of_seats | INT         | -                 | Total seats in this airplane        |
| Airplane_type     | VARCHAR(50) | FK → airline_type | Type classification of the airplane |

### 5.3 Table: airport

| Column Name  | Data Type    | Constraint  | Description                   |
|--------------|--------------|-------------|-------------------------------|
| Airport_code | VARCHAR(10)  | PRIMARY KEY | Unique airport code           |
| Name         | VARCHAR(255) | -           | Full name of the airport      |
| City         | VARCHAR(100) | -           | City where airport is located |
| State        | VARCHAR(100) | -           | State/province/region         |

### 5.4 Table: can\_land

| Column Name        | Data Type   | Constraint             | Description             |
|--------------------|-------------|------------------------|-------------------------|
| Airplane_type_name | VARCHAR(50) | PK + FK → airline_type | Aircraft type reference |
| Airport_code       | VARCHAR(10) | PK + FK → airport      | Airport reference       |

### 5.5 Table: flight

| Column Name   | Data Type   | Constraint  | Description              |
|---------------|-------------|-------------|--------------------------|
| Flight_number | VARCHAR(20) | PRIMARY KEY | Unique flight identifier |

| Column Name | Data Type    | Constraint | Description                                           |
|-------------|--------------|------------|-------------------------------------------------------|
| Airline     | VARCHAR(100) | -          | Operating airline name                                |
| Weekdays    | VARCHAR(50)  | -          | Days the flight operates (e.g., Mon,Wed,Fri or Daily) |

## 5.6 Table: flight\_leg

| Column Name              | Data Type   | Constraint       | Description                |
|--------------------------|-------------|------------------|----------------------------|
| Flight_number            | VARCHAR(20) | PK + FK → flight | Flight reference           |
| Leg_number               | INT         | PRIMARY KEY      | Sequence number of the leg |
| Departue_airport_code    | VARCHAR(10) | FK → airport     | Departure airport          |
| Scheduled_departure_time | TIME        | -                | Scheduled departure time   |
| Arrival_airport_code     | VARCHAR(10) | FK → airport     | Arrival airport            |
| Scheduled_arrval_time    | TIME        | -                | Scheduled arrival time     |

## 5.7 Table: leg\_instance

| Column Name            | Data Type   | Constraint           | Description                          |
|------------------------|-------------|----------------------|--------------------------------------|
| Flight_number          | VARCHAR(20) | PK + FK              | Flight reference                     |
| Leg_number             | INT         | PK + FK → flight_leg | Leg reference                        |
| Date                   | DATE        | PRIMARY KEY          | Operating date for this leg instance |
| No_of_available_seats  | INT         | -                    | Remaining available seats            |
| Airplane_id            | VARCHAR(20) | FK → airplane        | Assigned airplane                    |
| Departure_airport_code | VARCHAR(10) | FK → airport         | Actual departure airport             |
| Departure_time         | TIME        | -                    | Actual departure time                |
| Arrival_airport_code   | VARCHAR(10) | FK → airport         | Actual arrival airport               |
| Arrival_time           | TIME        | -                    | Actual arrival time                  |

## 5.8 Table: fare

| Column Name   | Data Type     | Constraint       | Description                     |
|---------------|---------------|------------------|---------------------------------|
| Flight_number | VARCHAR(20)   | PK + FK → flight | Flight reference                |
| Fare_code     | VARCHAR(20)   | PRIMARY KEY      | Fare class (ECONOMY / BUSINESS) |
| Amount        | DECIMAL(10,2) | -                | Ticket price                    |

| Column Name  | Data Type | Constraint | Description                                      |
|--------------|-----------|------------|--------------------------------------------------|
| Restrictions | TEXT      | -          | Booking restrictions (e.g., No Refund, Flexible) |

## 5.9 Table: seat\_reservation

| Column Name    | Data Type    | Constraint             | Description              |
|----------------|--------------|------------------------|--------------------------|
| Flight_number  | VARCHAR(20)  | PK + FK                | Flight reference         |
| Leg_number     | INT          | PK + FK                | Leg reference            |
| Date           | DATE         | PK + FK → leg_instance | Travel date              |
| Seat_no        | VARCHAR(10)  | PRIMARY KEY            | Seat number              |
| Customer_name  | VARCHAR(255) | -                      | Passenger full name      |
| Customer_phone | VARCHAR(15)  | -                      | Passenger contact number |

## 6. Foreign Key Constraints

The following foreign key relationships enforce referential integrity across the database:

| Table            | Column                          | References                        | On                      |
|------------------|---------------------------------|-----------------------------------|-------------------------|
| airplane         | Airplane_type                   | airline_type (Airplane_type_name) | airplane_ibfk_1         |
| can_land         | Airplane_type_name              | airline_type (Airplane_type_name) | can_land_ibfk_1         |
| can_land         | Airport_code                    | airport (Airport_code)            | can_land_ibfk_2         |
| fare             | Flight_number                   | flight (Flight_number)            | fare_ibfk_1             |
| flight_leg       | Flight_number                   | flight (Flight_number)            | flight_leg_ibfk_1       |
| flight_leg       | Departue_airport_code           | airport (Airport_code)            | flight_leg_ibfk_2       |
| flight_leg       | Arrival_airport_code            | airport (Airport_code)            | flight_leg_ibfk_3       |
| leg_instance     | Flight_number, Leg_number       | flight_leg (composite PK)         | leg_instance_ibfk_1     |
| leg_instance     | Airplane_id                     | airplane (Airplane_id)            | leg_instance_ibfk_2     |
| leg_instance     | Departure_airport_code          | airport (Airport_code)            | leg_instance_ibfk_3     |
| leg_instance     | Arrival_airport_code            | airport (Airport_code)            | leg_instance_ibfk_4     |
| seat_reservation | Flight_number, Leg_number, Date | leg_instance (composite PK)       | seat_reservation_ibfk_1 |

## 7. Sample Data Summary

The database was populated with realistic sample data to demonstrate all relationships and system functionality. The table below summarizes the number of records inserted per table.

| Table            | Records Inserted | Notes                                        |
|------------------|------------------|----------------------------------------------|
| airline_type     | 95               | Types 001–095, from Companies A–J            |
| airplane         | 95               | APL001–APL095, mapped 1-to-1 to types        |
| airport          | 99               | AP001–AP099, generic city/state names        |
| can_land         | 95               | One landing permission per type-airport pair |
| flight           | 95               | FL001–FL095, across 10 airlines              |
| flight_leg       | 99               | One leg per flight, covering FL001–FL099     |
| leg_instance     | 99               | Dated July 1–20, 2025                        |
| fare             | 198              | 2 fare codes per flight: ECONOMY & BUSINESS  |
| seat_reservation | 99               | One reservation per leg instance             |

## 8. Key Design Decisions

- **Composite Primary Keys:** The `flight_leg`, `leg_instance`, `fare`, and `seat_reservation` tables use composite primary keys to uniquely identify records that depend on multiple parent entities.
- **Separation of Flight and Leg Instance:** The `flight` and `flight_leg` tables define the static schedule, while `leg_instance` tracks each actual dated operation. This avoids data redundancy and follows good normalization principles.
- **Can Land Junction Table:** The many-to-many relationship between airplane types and airports is handled via the `can_land` bridge table, which is a standard approach for resolving M:N relationships in relational databases.
- **Fare Flexibility:** Storing fare codes as strings (ECONOMY, BUSINESS) rather than integers allows easy extensibility to add new fare classes in the future.
- **InnoDB Engine:** All tables use InnoDB, which supports foreign key constraints and ACID-compliant transactions — necessary for a system handling seat bookings and reservations.

## 9. Tools & Technologies

| Technology         | Details                       |
|--------------------|-------------------------------|
| Database           | MariaDB 10.4.11               |
| Management Tool    | phpMyAdmin 5.0.1              |
| Server Environment | XAMPP / localhost (127.0.0.1) |

| Technology     | Details                                         |
|----------------|-------------------------------------------------|
| Query Language | SQL (Structured Query Language)                 |
| Storage Engine | InnoDB (supports FK constraints & transactions) |
| Character Set  | utf8mb4 (full Unicode support)                  |

## 10. Conclusion

---

The Airline Management System database (pak\_airlines) successfully demonstrates the application of core DBMS concepts including entity-relationship modeling, normalization, primary/foreign key constraints, and data integrity enforcement.

The design supports the complete lifecycle of airline operations — from registering aircraft and airports, scheduling flights across multiple legs, tracking individual flight instances, managing fare pricing, and handling passenger seat reservations.

This project reinforces the practical importance of relational database design in building real-world information systems. The use of InnoDB with foreign key constraints ensures that all data remains consistent and that orphaned records cannot be created through invalid operations.